

1 . System Requirements Document (SRD)

Purpose

- Define what the system must do.
- Serve as the source for all software requirements.
- Provide traceable safety and functional requirements for certification.
- Include functional, performance, environmental, and safety aspects.

Inputs

- Customer requirements (monitor temperature, warn pilot)
- Operational Concept (ConOps)
- Safety assessment (FHA, PSSA, SSA)
- System architecture
- Interface Control Document (ICD)
- Performance and timing specifications
- Environmental requirements
- Regulatory and standard compliance
- Fault handling requirements

Example Requirements

- SYS-REQ- 0 1 : System shall monitor temperature sensor input every 1 second.
- SYS-REQ- 0 2 : System shall generate a warning when temperature exceeds 8 0 °C.
- SYS-REQ- 0 3 : System shall clear the warning when temperature drops below 7 5 °C.
- SYS-REQ- 0 4 : System shall store the over-temperature flag as Boolean.

Finalization

- Draft creation by system engineers
- Review by software, safety, QA
- Comments resolved
- Baseline under CM, version v 1 . 0 approved

Traceability

SRD	HLR
SYS-REQ- 0 1	HLR- 0 1

SRD	HLR
SYS-REQ- 0 2	HLR- 0 2

2 . High-Level Requirements Document (HLR)

Purpose

- Define what the software must do to satisfy SRD requirements.
- Translate system requirements into software-specific, testable statements.
- Include functional behavior, performance, and safety requirements.

Inputs

- SRD, ConOps, ICD
- Safety analysis data
- Performance and timing requirements
- Environmental constraints

Example HLR Requirements

- HLR- 0 1 : Software shall sample temperature sensor every 1 second.
- HLR- 0 2 : Software shall detect over-temperature when temp > 8 0 °C.
- HLR- 0 3 : Software shall clear over-temperature warning when temp < 7 5 °C.
- HLR- 0 4 : Software shall set an over-temperature warning flag as Boolean.

Finalization

- Peer review by software, verification, QA
- Verify completeness, clarity, and testability
- Comments resolved and baseline under CM

Traceability

SRD	HLR
SYS-REQ- 0 1	HLR- 0 1
SYS-REQ- 0 2	HLR- 0 2

3 . Low-Level Requirements Document (LLR)

Purpose

- Provide detailed, testable logic to implement HLR.
- Specify exact behavior, data structures, and interfaces.
- Serve as basis for code development and verification.

Inputs

- HLR, safety assessment, timing specifications
- ICD, environmental constraints, performance data

Example LLR Requirements

- LLR- 0 1 : Sample temperature sensor every 1 0 0 0 ms.
- LLR- 0 2 : If temperature > 8 0 °C for 2 consecutive samples, set warning flag TRUE.
- LLR- 0 3 : If temperature ≤ 7 5 °C for 2 consecutive samples, clear warning flag FALSE.
- LLR- 0 4 : Warning flag stored as Boolean variable.

Finalization

- Technical review and verification
- Ensure alignment with HLR
- Fix defects, baseline under CM

Traceability

HLR	LLR
HLR- 0 2	LLR- 0 2
HLR- 0 3	LLR- 0 3

4 . Software Design Description (SDD) – HLD and LLD

Purpose

- Provide High-Level Design (HLD) and Low-Level Design (LLD).
- HLD: Define modules, state machines, data flow, and major interfaces.

- LLD: Detail function logic, data structures, algorithms, and sequence diagrams.
- Serve as reference for coding and verification.

Inputs

- LLR, HLR, coding standards
- Hardware architecture, performance/timing specifications
- Safety and environmental constraints, ICD

High-Level Design (HLD)

- Module: TempMonitor
- Inputs: Temperature sensor signal
- Outputs: Warning flag
- States: NORMAL, OVER_TEMP
- Data Flow: Sensor → TempMonitor → Warning Flag
- Interfaces: Hardware registers, cockpit display, communication bus
- Description: Major modules, data flow, state transitions, responsibilities

Low-Level Design (LLD)

- Function: TempMonitor_Update()
- Input: Sensor value
- Logic:

```
if(sensor_value > 80 && prev_state == NORMAL) count++;
if(count >= 2) state = OVER_TEMP;
if(sensor_value <= 75 && prev_state == OVER_TEMP) count--;
if(count <= 0) state = NORMAL;
```

- Data Structures: state, count, warning_flag
- Interfaces: Sensor register, output flag to cockpit display
- Sequence Diagram: Sensor read → Logic processing → Flag set → Output

Finalization

- Design review by software, verification, QA
- Ensure all SRD and HLR requirements covered
- Fix comments, baseline under CM, approved v 1 . 0

Traceability

SRD	HLR	HLD Module	LLD Function
SYS-REQ- 0 1	HLR- 0 1	TempMonitor	TempMonitor_Update()

SRD	HLR	HLD Module	LLD Function
SYS-REQ- 0 2	HLR- 0 2	TempMonitor	TempMonitor_Update()
SYS-REQ- 0 3	HLR- 0 3	TempMonitor	TempMonitor_Update()

5 . Software Verification Cases & Procedures (SVCP)

Purpose

- Define test cases to verify HLR/LLR and design implementations.
- Include steps, inputs, and expected results.

Inputs

- HLR, LLR, HLD/LLD, coverage requirements

Example Test Cases

- TC- 0 1 : Temp = 6 0 °C → flag FALSE
- TC- 0 2 : Temp = 8 5 °C → flag TRUE
- TC- 0 3 : Temp = 7 0 °C → flag FALSE
- TC- 0 4 : Temp = 8 0 °C → flag FALSE (boundary)

Finalization

- Review by verification team + QA
- Ensure all requirements and design covered
- Baseline in CM

Traceability

LLR	Test Case
LLR- 0 2	TC- 0 2
LLR- 0 3	TC- 0 3

6 . Software Verification Results (SVR)

Purpose

- Record test execution results.
- Provide evidence that software satisfies requirements and design.

Inputs

- SVCP, logs, coverage reports, anomaly reports

Example Results

Test Case	Result
TC- 0 1	PASS
TC- 0 2	PASS
TC- 0 3	PASS
TC- 0 4	PASS
Coverage: Statement 1 0 0 %, Branch 1 0 0 %	

Finalization

- Review by verification lead + QA
- Fix open anomalies
- Baseline in CM

Traceability

Test Case	Result
TC- 0 2	PASS

7 . Software Validation Document (VDD / SVD)

Purpose

- Summarize that software meets operational requirements.
- Confirm all requirements tested and passed.

- Provide evidence for certification.

Inputs

- HLR, LLR, HLD/LLD, SVCP, SVR, traceability, coverage, anomaly reports

Content

- Validation strategy
- Summary of executed tests
- Coverage statistics
- Confirmation that all requirements validated
- No open anomalies

Finalization

- Review by verification + QA
- Baseline under CM
- Approved for certification

Traceability

SRD	HLR	LLR	HLD	LLD	Code	Test	Result
SYS-REQ- 0 2	HLR- 0 2	LLR- 0 2	TempMonitor	TempMonitor_Update()	TempMonitor()	TC- 0 2	PASS
SYS-REQ- 0 3	HLR- 0 3	LLR- 0 3	Reset Logic	TempMonitor_Update()	TempMonitor()	TC- 0 3	PASS

Final Traceability Chain

SRD → HLR → HLD → LLD → Code → SVCP → SVR → Software Validation Document